

Московский авиационный институт
(национальный исследовательский университет)
Факультет прикладной математики и информатики

Кафедра мехатроники и теоретической механики

Отчёт по курсовой работе

по курсу “Технология программирования на платформе .Net в механике и
мехатронике”

Выполнили: Меркулов М. А.
Хитин А. М.

Группа: М8О-102М-23

Преподаватель: Чекина Е.А.

Дата:

Оценка:

Подпись:

Постановка задачи

1) Смоделировать аттракцион в Unity, используя меши для создания отдельных цельных деталей механизма (можно создавать непосредственно в Unity, также возможно импортирование из стороннего 3d редактора) и различные виды соединений (joints) для совмещения их в один аттракцион.

2) Провести несколько экспериментов для исследования движения получившейся модели.

3) Написать отчет о проведенной работе.

Описание модели аттракциона

В качестве аттракциона для моделирования в Unity нами было выбрано колесо обозрения.

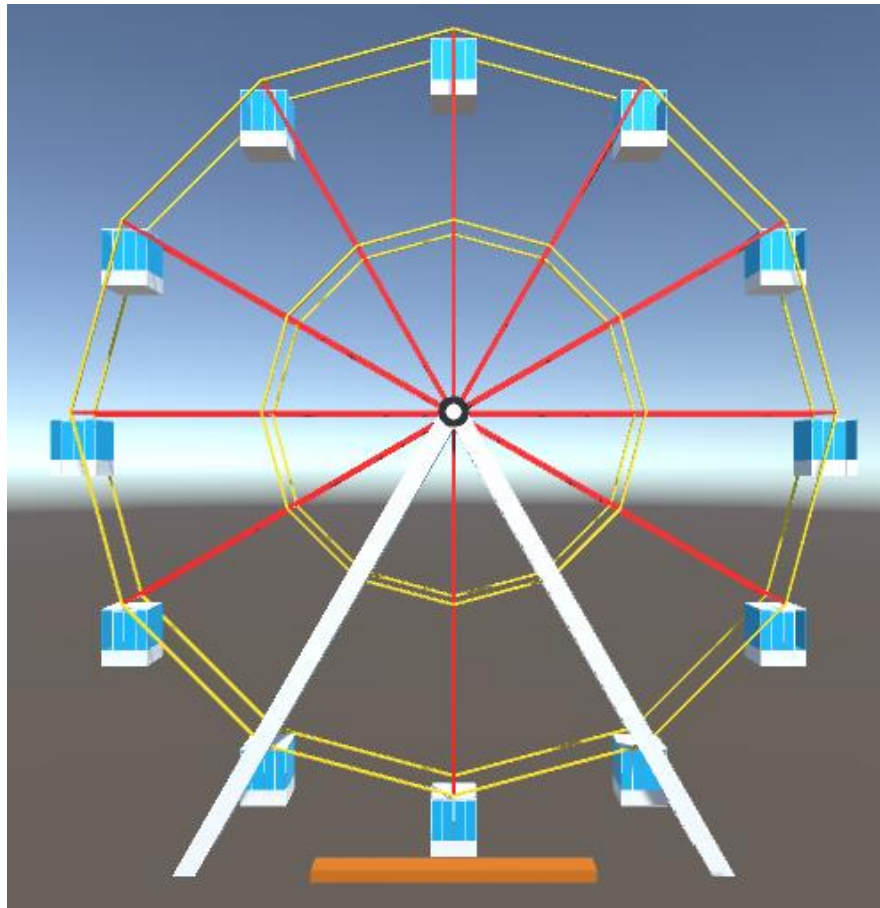


Рис. 1 Модель колеса обозрения

Моделирование колеса обозрения было полностью реализовано с помощью ScriptableWizard, поэтому мы можем сгенерировать модель колеса обозрения с различными параметрами без особых проблем. Мастер создания выглядит следующим образом:

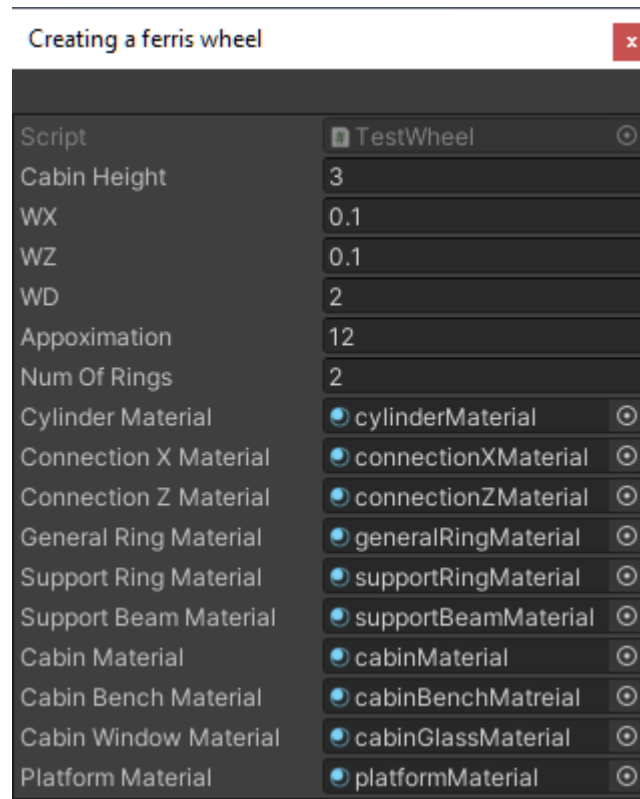


Рис. 2 Мастер создания модели колеса обозрения

Здесь

Cabin Height – высота кабинки аттракциона

WX – Толщина соединения по оси OX

WZ – Толщина соединения по оси OZ

WD – Расстояние между кольцами каркаса колеса обозрения по оси OZ

Approximation – Задаёт число точек, по которым строятся кольца каркаса колеса обозрения (влияет на число соединений между кольцами, а так же на число кабинок колеса обозрения)

Num of Rings – количество колец каркаса колеса обозрения

Cylinder Material – материал цилиндра, в центре колеса обозрения, соединяющего каркас колеса обозрения с опорами

Connection X Material – материал соединений, соединяющих кольца каркаса разных радиусов

Connection Z Material – материал соединений, соединяющих кольца каркаса одинаковых радиусов

General Ring Material – материал колец каркаса

Support Ring Material – материал кольца опоры, опоясывающего центральный цилиндр колеса обозрения

Support Beam Material – материал стоек опор

Cabin Material – основной материал кабинок

Cabin Bench Material – материал лавочек внутри кабинок

Cabin Window Material – материал окон кабинок

Platform Material – материал платформы для посадки/высадки пассажиров колеса обозрения

Ниже приведены несколько вариаций колеса обозрения в зависимости от заданных параметров:

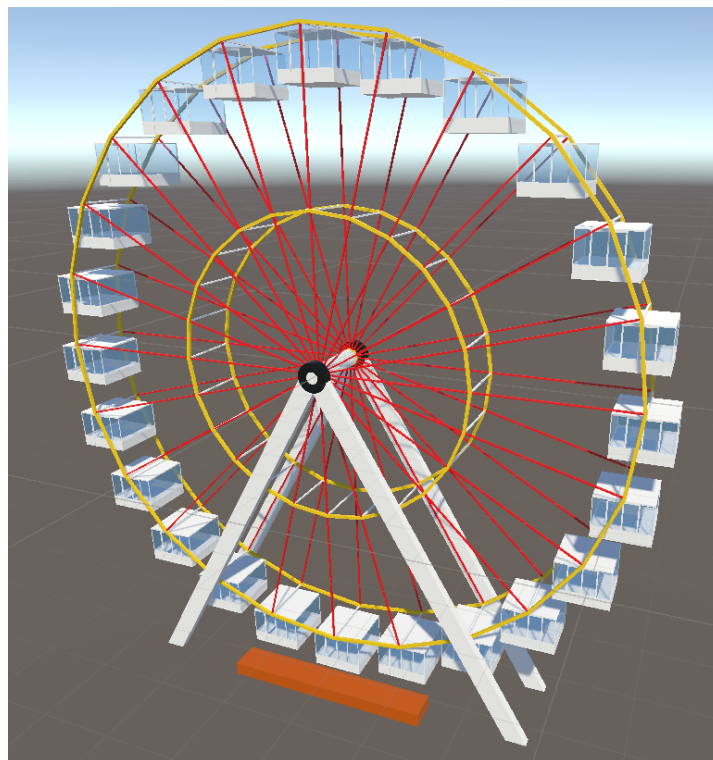


Рис. 3 Модель с высотой кабинки = 2 и параметром Approximation = 24

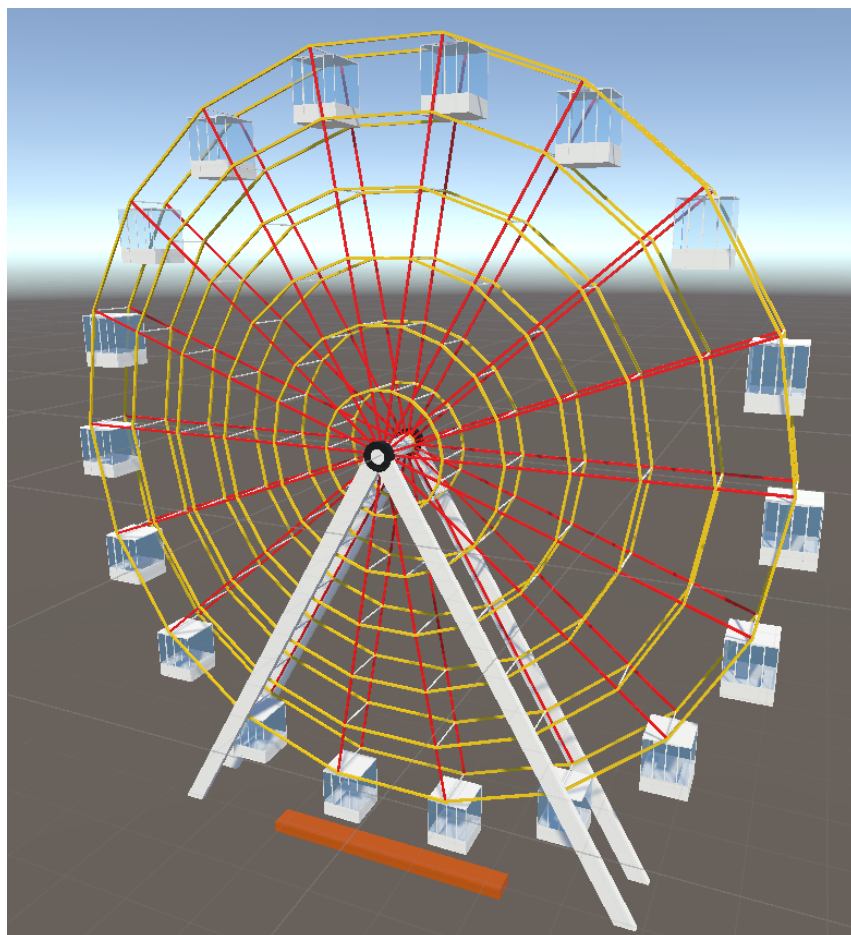


Рис. 4 Модель с параметром Num Of Rings = 6 и параметром Approximation = 18

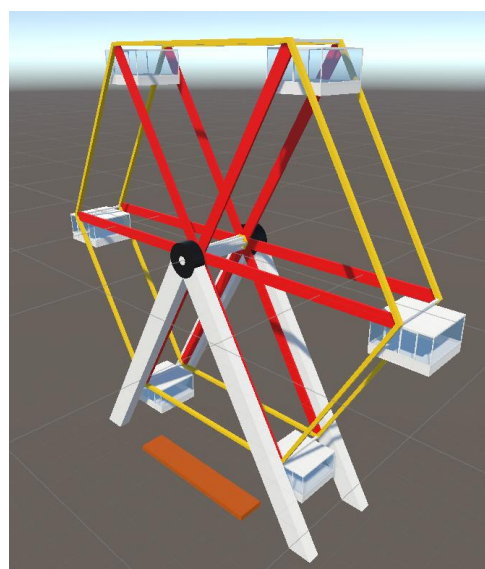


Рис. 5 Модель с параметрами $WX = WZ = 0.3$, Num Of Rings = 1 и параметром Approximation = 6

Конструкция аттракциона

Опишем основные части конструкции колеса обозрения, согласно параметрам мастера, описанным ранее:

1) Колесо обозрения имеет ($\text{Num of Rings} + 1$) основных колец каркаса различного радиуса (Кольца жёлтого цвета), добавочное кольцо опоясывает цилиндр, его вы можете видеть на рисунке 6

2) Основные кольца колеса обозрения разного радиуса соединяются красными соединениями стреловидной формы, их число будет зависеть от параметра `Approximation`

3) Между основными кольцами одного радиуса имеются белые соединения, представленные 4-угольными прямыми призмами, их количество также прямо зависит от параметра `Approximation`

4) В центре конструкции располагается белый цилиндр, он связан с точкой пространства в центре масс цилиндра с помощью соединения `Hinge Joint`. Данный цилиндр осуществляет вращение всего колеса обозрения при помощи мотора

5) 2 опоры, каждая из которых состоит из 2-ух опорных стоек белого цвета и чёрного кольца, опоясывающего центральный цилиндр колеса обозрения

6) Между внешними кольцами колеса обозрения располагаются пассажирские кабинки, каждая из которых привязана к середине соответствующего белого соединения между внешними кольцами. Число кабинок равно числу соединений, то есть их количество составляет `Approximation`. Помимо этого, к каждой кабинке привязан скрипт `DoorsScript`, который попадая в триггер, расположенный в нижней части колеса обозрения, осуществляет открытие/закрытие дверей кабинок. Пример кабинки, изображён на рис. 9, вид изнутри кабинки на рис. 10

7) Оранжевая платформа для посадки/высадки пассажиров колеса обозрения

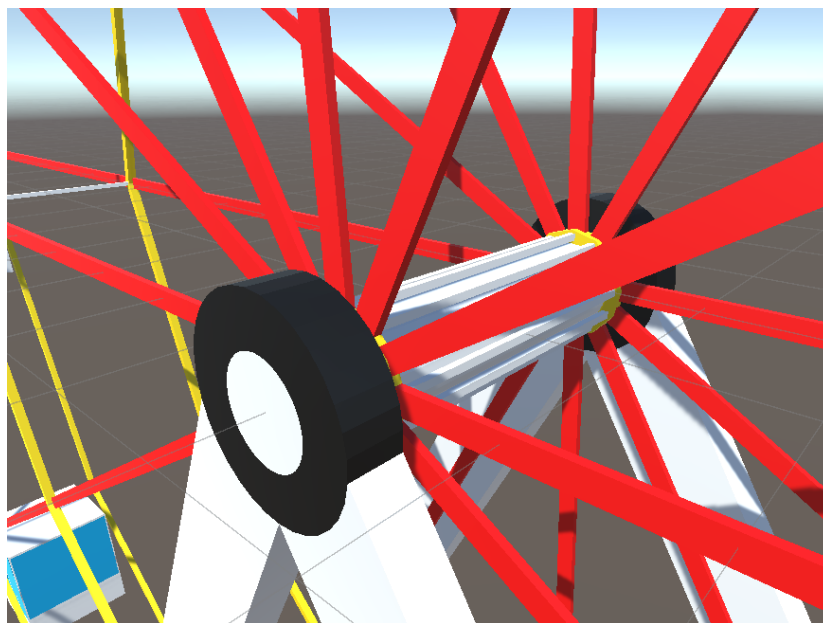


Рис. 6 Центр колеса обозрения

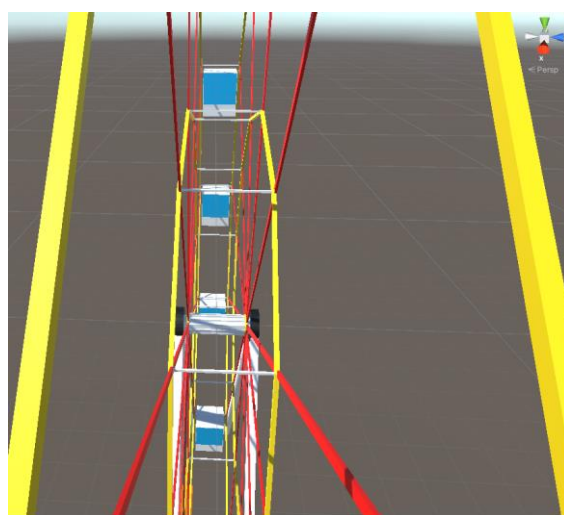
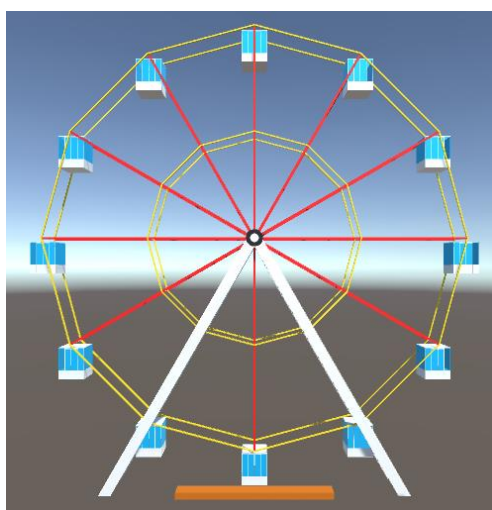


Рис. 7, 8 – Вид спереди, вид сбоку

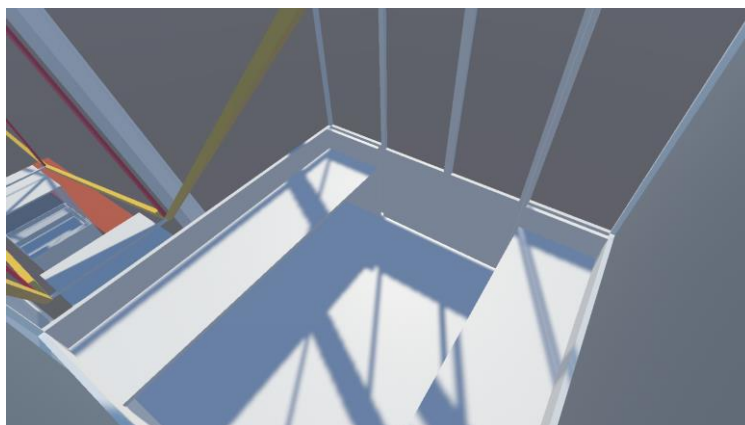
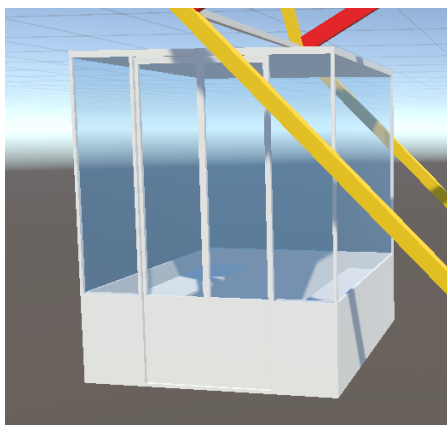
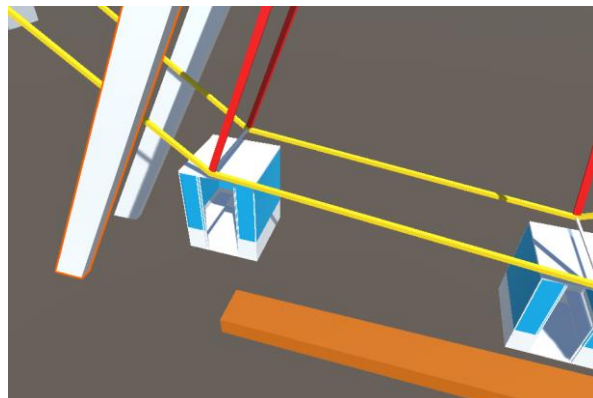
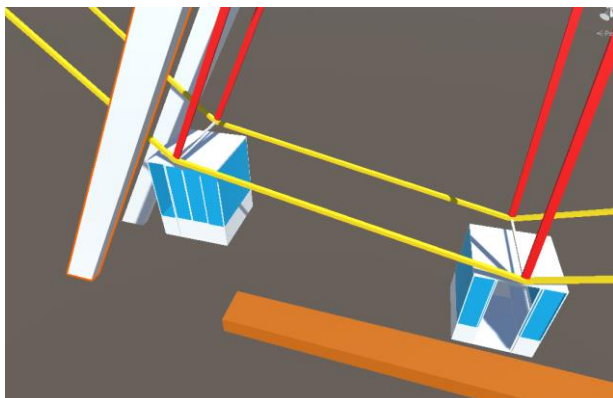
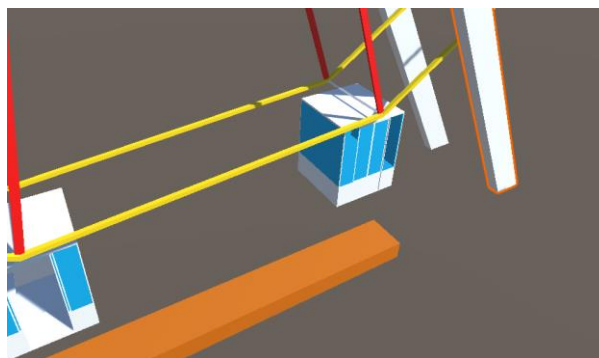
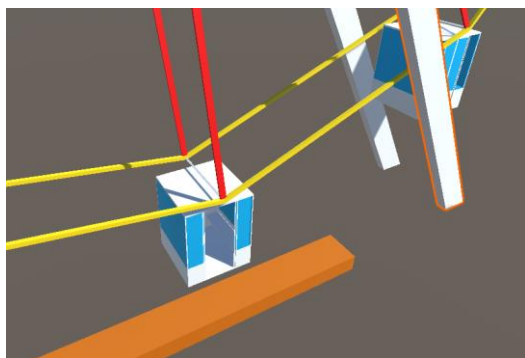


Рис. 9, 10 – Кабинка колеса обозрения, и вид изнутри кабинки

Когда кабинки проезжают рядом с платформой (то есть попадают в специальную триггерную зону), то происходит автоматическое открытие дверей.



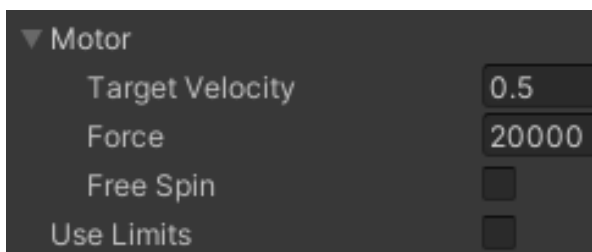
Как только кабинка покидает эту зону, то двери этой кабинки закрываются обратно.



Исследование движения получившейся модели

Нами было проведено исследование движения получившейся модели, параметры модели соответствуют рис. 2, добавочно, масса самого кольца обозрения в эксперименте равна 200 кг, масса каждой кабинки 2 кг.

Характеристики мотора, с помощью которого реализовано вращение колеса обозрения:



Колесо вращается со скоростью 0.5 м/с.

При заданных параметрах колесо обозрения совершает поступательное вращение, без каких-либо заметных отклонений

При увеличении массы кабинок до 50 кг, за счёт того, что на кабинки действует сила гравитации, и в данном примере их суммарная масса превышает массу колеса обозрения в 3 раза, наблюдается интересный эффект, при котором кабинка очень быстро раскручивается под собственным весом из-за конфликта действующих сил и перевеса опорной конструкции, что полностью ломает установленные физические связи, вы можете видеть данный эффект ниже

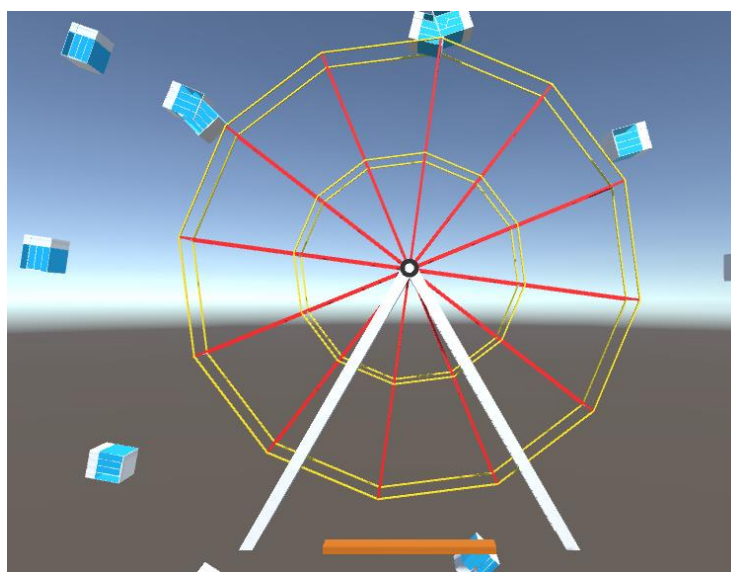


Рис. 11 – Нарушение связей с кабинками колеса обозрения

Также в процессе тестирования при массе колеса обозрения в 1 кг, и массе кабинок так же в 1 кг, наблюдается довольно странное явление: кабинки как бы подпрыгивают, причём, что интересно, те кабинки, угол расположения которых по тригонометрическому кругу больше подпрыгивают сильнее, данное явление странно тем, что вообще говоря ожидалось поведение, которое мы описали для предыдущего набора с массами 200 кг и 50 кг соответственно, но при малых массах оно наблюдается лишь при большем увеличении массы кабинок (явление повторилось при массе кольца в 1кг, и массе кабинки в 10 кг)

Скрипт DoorsScript для открытия и закрытия дверей кабинки

```
using System.Collections;
using System.Collections.Generic;
using System.Threading;
using UnityEngine;

public class DoorsScript : MonoBehaviour
{
    public GameObject leftDoor;
    public GameObject rightDoor;
    public float doorWidth;
    bool doorsClosed = true;
    int maxCount = 100;

    private void OnTriggerEnter(Collider other)
    {
        if (other.gameObject.tag == "OpenDoorsZone" && doorsClosed)
        {
            StartCoroutine(OpenDoors());
        }
    }
    private void OnTriggerExit(Collider other)
    {
        if (!doorsClosed)
        {
            StartCoroutine(CloseDoors());
        }
    }

    IEnumerator OpenDoors()
    {
        {
            int count = 0;
            doorsClosed = false;
            while (count < maxCount)
            {
                leftDoor.transform.Translate(-doorWidth / maxCount, 0, 0);
                rightDoor.transform.Translate(doorWidth / maxCount, 0, 0);
                count++;
                yield return new WaitForSeconds(2.5f / maxCount);
            }
        }
    }

    IEnumerator CloseDoors()
    {
        {
            int count = 0;
            doorsClosed = true;
            while (count < maxCount)
```

```

        {
            leftDoor.transform.Translate(doorWidth / maxCount, 0, 0);
            rightDoor.transform.Translate(-doorWidth / maxCount, 0, 0);
            count++;
            yield return new WaitForSeconds(2.5f / maxCount);
        }
    }
}

```

Скрипт TestWheel мастера колеса обозрения

```

using System.Collections;
using System.Collections.Generic;
using System.IO;
using System.Runtime.CompilerServices;
using Unity.VisualScripting;
using Unity.VisualScripting.FullSerializer;
using UnityEditor;
using UnityEngine;
using UnityEngine.Rendering;

public class TestWheel : ScriptableWizard
{
    public float cabinHeight = 3f; // Высота кабинки
    public float wX = 0.1f; // Толщина соединений по оси OX
    public float wZ = 0.1f; // Толщина соединений по оси OZ
    public float wD = 2f; // Расстояние между основными кольцами (по оси OZ)

    public int appoximation = 12;
    public int numOfRings = 2;
    int cylinderApproximation = 32;

    public Material cylinderMaterial;
    public Material connectionXMaterial;
    public Material connectionZMaterial;
    public Material generalRingMaterial;
    public Material supportRingMaterial;
    public Material supportBeamMaterial;

```

```

public Material cabinMaterial;

public Material cabinBenchMaterial;

public Material cabinWindowMaterial;

public Material platformMaterial;


[MenuItem("GameObject/3D Object/TestWheel")]
static void ShowConeWizard()
{
    ScriptableWizard.DisplayWizard<TestWheel>("Creating a ferris wheel", "Create");
}


void OnWizardCreate()
{
    GameObject main = new GameObject();
    main.name = "Ferris Wheel";


    float angleStep = 2 * Mathf.PI / appoximation;
    float cylinderAngleStep = 2 * Mathf.PI / cylinderApproximation;
    Material[] materials = new Material[]
    {
        // Части колеса обозрения
        cylinderMaterial,
        connectionXMaterial,
        connectionZMaterial,
        generalRingMaterial,
        // Части опор
        supportRingMaterial,
        supportBeamMaterial,
        // Части кабинки
        cabinMaterial,
        //Лавки
        cabinBenchMaterial,
        // Окна кабинки
        cabinWindowMaterial,
    }
}

```

```

// Материал платформы
platformMaterial
};

int simplePartCount = materials.Length;
List<Vector3>[] subVerticies = new List<Vector3>[simplePartCount];
List<int>[] subTriangles = new List<int>[simplePartCount];
List<Vector2>[] subUvs = new List<Vector2>[simplePartCount];
List<Mesh>[] singleMeshes = new List<Mesh>[simplePartCount];
List<bool>[] savedColliders = new List<bool>[simplePartCount];

for (int i = 0; i < simplePartCount; i++)
{
    subVerticies[i] = new List<Vector3>();
    subTriangles[i] = new List<int>();
    subUvs[i] = new List<Vector2>();
    singleMeshes[i] = new List<Mesh>();
    savedColliders[i] = new List<bool>();
}

List<Vector3> verticies = new List<Vector3>();
List<int> triangles = new List<int>();
List<Vector2> uvs = new List<Vector2>();

void ClearMesh()
{
    verticies = new List<Vector3>();
    triangles = new List<int>();
    uvs = new List<Vector2>();
}

void ClearSavedMesh(int saveIndex)
{
    subVerticies[saveIndex] = new List<Vector3>();
    subTriangles[saveIndex] = new List<int>();
}

```

```

        subUvs[saveIndex] = new List<Vector2>();
        singleMeshes[saveIndex] = new List<Mesh>();
        savedColliders[saveIndex] = new List<bool>();
    }

    void AddRectPoly(Vector3 point1, Vector3 point2, Vector3 point3, Vector3 point4)
    {
        verticies.AddRange(new Vector3[] { point1, point2, point3, point4 });
        uvs.AddRange(new Vector2[] { new Vector2(0, 0), new Vector2(1, 0), new Vector2(1, 1), new
Vector2(0, 1) });
        triangles.AddRange(new int[] { verticies.Count - 4, verticies.Count - 1, verticies.Count - 2,
verticies.Count - 2, verticies.Count - 3, verticies.Count - 4 });
    }

    void AddTriagPoly(Vector3 point1, Vector3 point2, Vector3 point3)
    {
        verticies.AddRange(new Vector3[] { point1, point2, point3 });
        uvs.AddRange(new Vector2[] { new Vector2(0, 0), new Vector2(1, 0), new Vector2(0, 1) });
        triangles.AddRange(new int[] { verticies.Count - 3, verticies.Count - 2, verticies.Count - 1 });
    }

    void AddPart(int saveIndex, bool saveCollider)
    {
        Mesh mesh = new Mesh();
        mesh.vertices = verticies.ToArray();
        mesh.triangles = triangles.ToArray();
        mesh.RecalculateNormals();
        mesh.uv = uvs.ToArray();

        singleMeshes[saveIndex].Add(mesh);
        savedColliders[saveIndex].Add(saveCollider);

        for(int i = 0; i < uvs.Count; i++)
        {
            subUvs[saveIndex].Add(uvs[i]);
        }
    }

```

```

    }

    for(int i = 0; i < triangles.Count;i++)
    {
        subTriangles[saveIndex].Add(triangles[i] + subVertices[saveIndex].Count);
    }

    for(int i = 0; i < verticies.Count;i++)
    {
        subVerticies[saveIndex].Add(verticies[i]);
    }
}

bool[] rectPolys = new bool[6] { true, true, true, true, true, true };
Vector3 d1 = Vector3.zero;
Vector3 d2 = Vector3.zero;
Vector3 posCenter;
Vector3 negCenter;
void AddRectFigXY(int saveIndex, bool saveCollider)
{
    Vector3 A1 = new Vector3(posCenter.x + d1.x, posCenter.y + d1.y, posCenter.z + d1.z);
    Vector3 B1 = new Vector3(posCenter.x - d1.x, posCenter.y - d1.y, A1.z);
    Vector3 C1 = new Vector3(B1.x, B1.y, posCenter.z - d1.z);
    Vector3 D1 = new Vector3(A1.x, A1.y, C1.z);

    Vector3 A2 = new Vector3(negCenter.x + d2.x, negCenter.y + d2.y, negCenter.z + d2.z);
    Vector3 B2 = new Vector3(negCenter.x - d2.x, negCenter.y - d2.y, A2.z);
    Vector3 C2 = new Vector3(B2.x, B2.y, negCenter.z - d2.z);
    Vector3 D2 = new Vector3(A2.x, A2.y, C2.z);

    if (rectPolys[0])
        AddRectPoly(A1, D1, C1, B1);
    if (rectPolys[1])
        AddRectPoly(A1, B1, B2, A2);

```



```

    if (rectPolys[2])
        AddRectPoly(B1, C1, C2, B2);
    if (rectPolys[3])
        AddRectPoly(C1, D1, D2, C2);
    if (rectPolys[4])
        AddRectPoly(D1, A1, A2, D2);
    if (rectPolys[5])
        AddRectPoly(A2, B2, C2, D2);

    AddPart(saveIndex, saveCollider);
}

```

```

void AddRectFigXZ(int saveIndex, bool saveCollider)
{
    Vector3 A1 = new Vector3(posCenter.x + d1.x, posCenter.y + d1.y, posCenter.z);
    Vector3 B1 = new Vector3(A1.x, posCenter.y - d1.y, A1.z);
    Vector3 C1 = new Vector3(posCenter.x - d1.x, B1.y, A1.z);
    Vector3 D1 = new Vector3(C1.x, A1.y, A1.z);

    Vector3 A2 = new Vector3(negCenter.x + d2.x, negCenter.y + d2.y, negCenter.z);
    Vector3 B2 = new Vector3(A2.x, negCenter.y - d2.y, A2.z);
    Vector3 C2 = new Vector3(negCenter.x - d2.x, B2.y, A2.z);
    Vector3 D2 = new Vector3(C2.x, A2.y, A2.z);

    if (rectPolys[0])
        AddRectPoly(A1, D1, C1, B1);
    if (rectPolys[1])
        AddRectPoly(A1, B1, B2, A2);
    if (rectPolys[2])
        AddRectPoly(B1, C1, C2, B2);
    if (rectPolys[3])
        AddRectPoly(C1, D1, D2, C2);
    if (rectPolys[4])
        AddRectPoly(D1, A1, A2, D2);
}

```

```

    if (rectPolys[5])
        AddRectPoly(A2, B2, C2, D2);

    AddPart(saveIndex, saveCollider);
}

GameObject CreateGameObject(int startIndex, int endIndex, string name, bool needRigidbody)
{
    Mesh mesh = new Mesh();
    List<Vector3> helperVertices = new List<Vector3>();
    List<int> helperTriangles = new List<int>();
    List<Vector2> helperUvs = new List<Vector2>();
    for (int i = startIndex; i <= endIndex; i++)
    {
        int vertCount = helperVertices.Count;
        for (int j = 0; j < subVertices[i].Count; j++)
        {
            helperVertices.Add(subVertices[i][j]);
        }
        for (int j = 0; j < subTriangles[i].Count; j++)
        {
            helperTriangles.Add(subTriangles[i][j] + vertCount);
        }
        for (int j = 0; j < subUvs[i].Count; j++)
        {
            helperUvs.Add(subUvs[i][j]);
        }
    }
    mesh.vertices = helperVertices.ToArray();
    mesh.triangles = helperTriangles.ToArray();
    mesh.RecalculateNormals();
    mesh.triangles = null;
    mesh.uv = helperUvs.ToArray();
}

```

MeshUpdateFlags	flags	=	MeshUpdateFlags.DontValidateIndices	
MeshUpdateFlags.DontResetBoneBounds			MeshUpdateFlags.DontNotifyMeshUsers	
MeshUpdateFlags.DontRecalculateBounds;				

```
mesh.SetIndexBufferParams(helperTriangles.Count, IndexFormat.UInt32);
mesh.SetIndexBufferData(helperTriangles.ToArray(), 0, 0, helperTriangles.Count, flags);
```

```
mesh.subMeshCount = endIndex - startIndex + 1;
int curIndex = 0;
for(int i = startIndex; i <= endIndex; i++)
{
    SubMeshDescriptor sMD = new SubMeshDescriptor(curIndex, subTriangles[i].Count);
    mesh.SetSubMesh(i - startIndex, sMD, flags);
    curIndex += subTriangles[i].Count;
}
```

```
GameObject gmObj = new GameObject();
gmObj.name = name;
MeshFilter meshf = gmObj.AddComponent<MeshFilter>();
MeshRenderer mr = gmObj.AddComponent<MeshRenderer>();
```

```
Material[] helperMaterials = new Material[endIndex - startIndex + 1];
for (int i = 0; i <= endIndex - startIndex; i++)
{
    helperMaterials[i] = materials[startIndex + i];
}
```

```
meshf.sharedMesh = mesh;
mr.materials = helperMaterials;
//meshf.mesh = mesh;
//mr.material = cylinderMaterial;
```

```
if (needRigidbody)
{
    gmObj.AddComponent<Rigidbody>();
}
```

```

        gmObj.GetComponent<Rigidbody>().useGravity = false;
    }

    for(int i = startIndex; i <= endIndex; i++)
    {
        for(int j = 0; j < singleMeshes[i].Count; j++)
        {
            if (!savedColliders[i][j])
            {
                continue;
            }

            MeshCollider collider = gmObj.AddComponent<MeshCollider>();

            collider.sharedMesh = singleMeshes[i][j];
            collider.convex = true;
        }
    }

    gmObj.transform.SetParent(main.transform);

    return gmObj;
}

// Creating Ferris wheel's center cylinder
float cR = cabinHeight * 0.1f;
float sRO = cR + 0f; // supportRingOffset = CylinderRadius + offset for motion of innerPart
float wSX = 3 * wX;
float ringsDist = 2 * wZ;
float cH = (wD + 2 * wZ + 2 * wSX + ringsDist);
//float centerZ = -(wD + 2 * wZ) / 2f;
posCenter = new Vector3(0, 0, cH / 2f);
negCenter = new Vector3(0, 0, -cH / 2f);

float alpha = Mathf.PI / 6;

```

```

float floorDist = cabinHeight * 6f;

//int numOfRings = 2;

float ringSize = ((floorDist - cabinHeight) / numOfRings);

Vector2 direction = new Vector2(Mathf.Cos(-Mathf.PI / 2f - alpha), Mathf.Sin(-Mathf.PI / 2f - alpha));

ClearMesh();

bool saveCollider = true;

for (int i = 0; i < cylinderApproximation; i++)
{
    Vector3 A = new Vector3(cR * Mathf.Cos(i * cylinderAngleStep), cR * Mathf.Sin(i *
cylinderAngleStep), posCenter.z);

    Vector3 B = new Vector3(A.x, A.y, negCenter.z);

    Vector3 C = new Vector3(cR * Mathf.Cos((i + 1) * cylinderAngleStep), cR * Mathf.Sin((i + 1) *
cylinderAngleStep), negCenter.z);

    Vector3 D = new Vector3(C.x, C.y, posCenter.z);

    // Грань B-A-D-C
    AddRectPoly(B, A, D, C);

    // Лицевая грань
    AddTriagPoly(A, D, posCenter);

    // Задняя грань
    AddTriagPoly(C, B, negCenter);
}

AddPart(0, saveCollider);

// Создаём соединения по оси X (ConnectionX)
rectPolys = new bool[6] { false, true, false, true, true, false };
saveCollider = false;

for (int curR = 0; curR < numOfRings; curR++)
{
    float wR1 = Mathf.Max(curR * ringSize, cR);

    float wR2 = (curR + 1) * ringSize;

    for (int i = 0; i < appoximation; i++)

```

```

{
    Vector2 p1 = new Vector2(cabinHeight * Mathf.Cos((i - 1) * angleStep), cabinHeight *
Mathf.Sin((i - 1) * angleStep));

    Vector2 p2 = new Vector2(cabinHeight * Mathf.Cos(i * angleStep), cabinHeight * Mathf.Sin(i
* angleStep));

    Vector2 p3 = new Vector2(cabinHeight * Mathf.Cos((i + 1) * angleStep), cabinHeight *
Mathf.Sin((i + 1) * angleStep));

    Vector3 center1 = new Vector3((wR1 + wX) * Mathf.Cos(i * angleStep), (wR1 + wX) *
Mathf.Sin(i * angleStep), -(wD + wZ) / 2f);

    Vector3 center2 = new Vector3(wR2 * Mathf.Cos(i * angleStep), wR2 * Mathf.Sin(i *
angleStep), center1.z);

    //Lower1
    ClearMesh();

    d1 = new Vector3((p1 - p2).x * wX / 4f, (p1 - p2).y * wX / 4f, wZ / 4f);
    d2 = d1;
    posCenter = center1 + d1;
    negCenter = center2 + d2;
    AddRectFigXY(1, saveCollider);

    //Upper1
    ClearMesh();

    d1 = new Vector3((p3 - p2).x * wX / 4f, (p3 - p2).y * wX / 4f, wZ / 4f);
    d2 = d1;
    posCenter = center2 + d2;
    negCenter = center1 + d1;
    AddRectFigXY(1, saveCollider);

    center1.z *= -1;
    center2.z = center1.z;

    //Lower2
    ClearMesh();

    d1 = new Vector3((p1 - p2).x * wX / 4f, (p1 - p2).y * wX / 4f, wZ / 4f);
    d2 = d1;
    posCenter = center1 + d1;

```

```

negCenter = center2 + d2;
AddRectFigXY(1, saveCollider);

//Upper2
ClearMesh();
d1 = new Vector3((p3 - p2).x * wX / 4f, (p3 - p2).y * wX / 4f, wZ / 4f);
d2 = d1;
posCenter = center2 + d2;
negCenter = center1 + d1;
AddRectFigXY(1, saveCollider);
}
}

// Создаём соединения по оси Z (ConnectionZ) и основные кольца (generalRings)
rectPolys = new bool[6] { false, true, true, true, true, false };
saveCollider = false;
for (int curR = 0; curR <= numOfRings; curR++)
{
    float wR = Mathf.Max(curR * ringSize, cR);
    for (int i = 0; i < appoximation; i++)
    {
        if (curR == numOfRings)
        {
            saveCollider = true;
        }
        // Создаём соединения по оси Z (ConnectionZ)
        ClearMesh();
        d1 = new Vector3(wX / 4f, wX / 4f, 0);
        d2 = d1;
        posCenter = new Vector3((wR + 2 * d1.x) * Mathf.Cos(i * angleStep), (wR + 2 * d1.y) *
        Mathf.Sin(i * angleStep), -wD / 2f);
        negCenter = new Vector3(posCenter.x, posCenter.y, -posCenter.z);
        AddRectFigXZ(2, saveCollider);
    }
}

```

```

        saveCollider = false;

        // Создаём первое кольцо

        ClearMesh();

        d1 = new Vector3(wX / 2f * Mathf.Cos(i * angleStep), wX / 2f * Mathf.Sin(i * angleStep), wZ /
2f);

        d2 = new Vector3(wX / 2f * Mathf.Cos((i + 1) * angleStep), wX / 2f * Mathf.Sin((i + 1) *
angleStep), wZ / 2f);

        posCenter = new Vector3((wR + wX / 2f) * Mathf.Cos(i * angleStep), (wR + wX / 2f) *
Mathf.Sin(i * angleStep), -(wD + wZ) / 2f);

        negCenter = new Vector3((wR + wX / 2f) * Mathf.Cos((i + 1) * angleStep), (wR + wX / 2f) *
Mathf.Sin((i + 1) * angleStep), posCenter.z);

        AddRectFigXY(3, saveCollider);

        // Создаём второе кольцо

        ClearMesh();

        posCenter.z *= -1;
        negCenter.z *= -1;

        AddRectFigXY(3, saveCollider);
    }
}

int sIndex = 0;
int eIndex = 3;

GameObject ferrisWheel = CreateGameObject(sIndex, eIndex, "Ferris Wheel", true);
Rigidbody rb = ferrisWheel.GetComponent<Rigidbody>();
rb.centerOfMass = new Vector3(0, 0, 0);

JointMotor motor = new JointMotor();
motor.targetVelocity = 0.5f;
motor.force = 20000f;

HingeJoint hj = ferrisWheel.AddComponent<HingeJoint>();
hj.anchor = new Vector3(0, 0, 0);
hj.axis = new Vector3(0, 0, 1);
hj.motor = motor;
hj.useMotor = true;

```



```

// Создаём первую опору
// Создаём кольцо опоры (supportRing)
rectPolys = new bool[6] { false, true, true, true, true, false };
saveCollider = false;
for(int i = 0; i < cylinderApproximation; i++)
{
    ClearMesh();
    float ringR = sRO + wSX / 2f;
    float curAngle = i * cylinderAngleStep;
    float nextAngle = (i + 1) * cylinderAngleStep;
    d1 = new Vector3((wSX / 2f) * Mathf.Cos(curAngle), (wSX / 2f) * Mathf.Sin(curAngle), wSX /
2f);
    d2 = new Vector3((wSX / 2f) * Mathf.Cos(nextAngle), (wSX / 2f) * Mathf.Sin(nextAngle), wSX /
2f);
    posCenter = new Vector3(ringR * Mathf.Cos(curAngle), ringR * Mathf.Sin(curAngle), -(wD +
wSX) / 2f - ringsDist);
    negCenter = new Vector3(ringR * Mathf.Cos(nextAngle), ringR * Mathf.Sin(nextAngle),
posCenter.z);
    AddRectFigXY(4, saveCollider);
}

// Создаём балки опоры
rectPolys = new bool[6] { false, true, false, true, true, false };
saveCollider = false;
for (int i = cylinderApproximation / 2 + 1; i < 3 * cylinderApproximation / 4; i++)
{
    ClearMesh();
    rectPolys[2] = false;
    rectPolys[4] = true;
    float ringR = sRO + wSX;
    float curAngle = i * cylinderAngleStep;
    float nextAngle = (i + 1) * cylinderAngleStep;
    Vector2 oldXY1 = new Vector2(ringR * Mathf.Cos(curAngle), ringR * Mathf.Sin(curAngle)); //, -
(wD + wSX) / 2f - ringsDist);
    Vector2 newXY1 = oldXY1 + (floorDist + oldXY1.y) / Mathf.Cos(alpha) * direction;

```

```

posCenter = new Vector3((oldXY1.x + newXY1.x) / 2f, (oldXY1.y + newXY1.y) / 2f, -(wD +
wSX) / 2f - ringsDist);

Vector2 oldXY2 = new Vector2(ringR * Mathf.Cos(nextAngle), ringR * Mathf.Sin(nextAngle));
Vector2 newXY2 = oldXY2 + (floorDist + oldXY2.y) / Mathf.Cos(alpha) * direction;

negCenter = new Vector3((oldXY2.x + newXY2.x) / 2f, (oldXY2.y + newXY2.y) / 2f,
posCenter.z);

d1 = new Vector3((newXY1.x - oldXY1.x) / 2f, (newXY1.y - oldXY1.y) / 2f, wSX / 2f);
d2 = new Vector3((newXY2.x - oldXY2.x) / 2f, (newXY2.y - oldXY2.y) / 2f, wSX / 2f);

if (i == cylinderApproximation / 2 + 1)
{
    rectPolys[0] = true;
    AddRectFigXY(5, saveCollider);
    rectPolys[0] = false;

    float colliderSize = 2 * cabinHeight;

    BoxCollider collider = main.AddComponent<BoxCollider>();
    main.tag = "OpenDoorsZone";
    collider.center = new Vector3(0, -floorDist + colliderSize / 2f, 0);
    collider.size = new Vector3(Mathf.Abs(newXY1.x), colliderSize, wD);
    collider.isTrigger = true;
}
else if (i == 3 * cylinderApproximation / 4 - 1)
{
    rectPolys[5] = true;
    AddRectFigXY(5, saveCollider);
    rectPolys[5] = false;
}
else
{
    AddRectFigXY(5, saveCollider);
}

ClearMesh();

rectPolys[2] = true;

```

```

rectPolys[4] = false;
posCenter = new Vector3(-posCenter.x, posCenter.y, posCenter.z);
negCenter = new Vector3(-negCenter.x, negCenter.y, negCenter.z);
d1 = new Vector3(d1.x, -d1.y, d1.z);
d2 = new Vector3(d2.x, -d2.y, d2.z);
if (i == cylinderApproximation / 2 + 1)
{
    rectPolys[0] = true;
    AddRectFigXY(5, saveCollider);
    rectPolys[0] = false;
}
else if (i == 3 * cylinderApproximation / 4 - 1)
{
    rectPolys[5] = true;
    AddRectFigXY(5, saveCollider);
    rectPolys[5] = false;
}
else
{
    AddRectFigXY(5, saveCollider);
}
}

sIndex = 4;
eIndex = 5;
GameObject supportingBeam1 = CreateGameObject(sIndex, eIndex, "supportingBeam1", false);

for(int i = sIndex; i <= eIndex; i++)
{
    ClearSavedMesh(i);
}

// Создаём вторую опору
// Создаём кольцо опоры (supportRing)

```

```

rectPolys = new bool[6] { false, true, true, true, true, false };
saveCollider = false;
for (int i = 0; i < cylinderApproximation; i++)
{
    ClearMesh();

    float ringR = sRO + wSX / 2f;

    float curAngle = i * cylinderAngleStep;

    float nextAngle = (i + 1) * cylinderAngleStep;

    d1 = new Vector3((wSX / 2f) * Mathf.Cos(curAngle), (wSX / 2f) * Mathf.Sin(curAngle), wSX /
2f);

    d2 = new Vector3((wSX / 2f) * Mathf.Cos(nextAngle), (wSX / 2f) * Mathf.Sin(nextAngle), wSX /
2f);

    posCenter = new Vector3(ringR * Mathf.Cos(curAngle), ringR * Mathf.Sin(curAngle), (wD +
wSX) / 2f + ringsDist);

    negCenter = new Vector3(ringR * Mathf.Cos(nextAngle), ringR * Mathf.Sin(nextAngle),
posCenter.z);

    AddRectFigXY(4, saveCollider);
}

// Создаём балки опоры
rectPolys = new bool[6] { false, true, false, true, true, false };
for (int i = cylinderApproximation / 2 + 1; i < 3 * cylinderApproximation / 4; i++)
{
    ClearMesh();

    rectPolys[2] = false;

    rectPolys[4] = true;

    float ringR = sRO + wSX;

    float curAngle = i * cylinderAngleStep;

    float nextAngle = (i + 1) * cylinderAngleStep;

    Vector2 oldXY1 = new Vector2(ringR * Mathf.Cos(curAngle), ringR * Mathf.Sin(curAngle)); //, -
(wD + wSX) / 2f - ringsDist);

    Vector2 newXY1 = oldXY1 + (floorDist + oldXY1.y) / Mathf.Cos(alpha) * direction;

    posCenter = new Vector3((oldXY1.x + newXY1.x) / 2f, (oldXY1.y + newXY1.y) / 2f, (wD + wSX)
/ 2f + ringsDist);

    Vector2 oldXY2 = new Vector2(ringR * Mathf.Cos(nextAngle), ringR * Mathf.Sin(nextAngle));

    Vector2 newXY2 = oldXY2 + (floorDist + oldXY2.y) / Mathf.Cos(alpha) * direction;

```

```

    negCenter = new Vector3((oldXY2.x + newXY2.x) / 2f, (oldXY2.y + newXY2.y) / 2f,
posCenter.z);

    d1 = new Vector3((newXY1.x - oldXY1.x) / 2f, (newXY1.y - oldXY1.y) / 2f, wSX / 2f);
    d2 = new Vector3((newXY2.x - oldXY2.x) / 2f, (newXY2.y - oldXY2.y) / 2f, wSX / 2f);
    if (i == cylinderApproximation / 2 + 1)
    {
        rectPolys[0] = true;
        AddRectFigXY(5, saveCollider);
        rectPolys[0] = false;
    }
    else if (i == 3 * cylinderApproximation / 4 - 1)
    {
        rectPolys[5] = true;
        AddRectFigXY(5, saveCollider);
        rectPolys[5] = false;
    }
    else
    {
        AddRectFigXY(5, saveCollider);
    }

    ClearMesh();
    rectPolys[2] = true;
    rectPolys[4] = false;
    posCenter = new Vector3(-posCenter.x, posCenter.y, posCenter.z);
    negCenter = new Vector3(-negCenter.x, negCenter.y, negCenter.z);
    d1 = new Vector3(d1.x, -d1.y, d1.z);
    d2 = new Vector3(d2.x, -d2.y, d2.z);
    if (i == cylinderApproximation / 2 + 1)
    {
        rectPolys[0] = true;
        AddRectFigXY(5, saveCollider);
        rectPolys[0] = false;
    }

```

```

else if (i == 3 * cylinderApproximation / 4 - 1)
{
    rectPolys[5] = true;
    AddRectFigXY(5, saveCollider);
    rectPolys[5] = false;
}
else
{
    AddRectFigXY(5, saveCollider);
}
}

sIndex = 4;
eIndex = 5;
GameObject supportingBeam2 = CreateGameObject(sIndex, eIndex, "supportingBeam2", false);

GameObject cabins = new GameObject();
cabins.name = "Cabins";
cabins.transform.SetParent(main.transform);

// Создаём кабинки
// Создаём пол кабинок
rectPolys = new bool[6] { true, true, true, true, true, true };
saveCollider = true;
float actualCabinHeight = cabinHeight - 0.5f;
float actualCabinWidth = 0.9f * wD / 2f;
float actualCabinThickness = 0.05f * actualCabinWidth / 2f;
float supR = numOfRings * ringSize;
float[] upperOffset = new float[]
{
    -actualCabinHeight + actualCabinThickness,
    -0.75f * actualCabinHeight,
    -0.1f * actualCabinHeight - 2 * actualCabinThickness,
    -0.1f * actualCabinHeight,

```

```

Of
};
float[] lowerOffset = new float[]
{
    -actualCabinHeight,
    upperOffset[0],
    upperOffset[1],
    upperOffset[2],
    upperOffset[3]
};

for (int i = 0; i < approximation; i++)
{
    sIndex = 6;
    eIndex = 8;//8;
    // Пол кабинки
    ClearMesh();
    d1 = new Vector3(actualCabinWidth, 0, actualCabinWidth);
    d2 = d1;
    Vector3 topCenter = new Vector3((supR + wX / 4f) * Mathf.Cos(i * angleStep), (supR + wX / 4f)
* Mathf.Sin(i * angleStep), 0);

    posCenter = new Vector3(topCenter.x, topCenter.y + lowerOffset[0], topCenter.z);
    negCenter = new Vector3(posCenter.x, topCenter.y + upperOffset[0], posCenter.z);
    AddRectFigXY(6, saveCollider);

    // Стенки под окнами (плоскость входа)
    ClearMesh();
    d1 = new Vector3(actualCabinWidth / 4f - actualCabinThickness, (upperOffset[1] - lowerOffset[1])
/ 2f, 0);
    d2 = d1;
    posCenter = new Vector3(topCenter.x - 3f * actualCabinWidth / 4f, topCenter.y + lowerOffset[1]
+ d1.y, topCenter.z - actualCabinWidth);
    negCenter = new Vector3(posCenter.x, posCenter.y, posCenter.z + actualCabinThickness);
    AddRectFigXZ(6, saveCollider);

```

```
ClearMesh();  
posCenter.x = topCenter.x + 3 * actualCabinWidth / 4f;  
negCenter.x = posCenter.x;  
AddRectFigXZ(6, saveCollider);
```

```
ClearMesh();  
posCenter.z = topCenter.z - actualCabinWidth + 2 * actualCabinThickness;  
negCenter.z = posCenter.z + actualCabinThickness;  
AddRectFigXZ(6, saveCollider);
```

```
ClearMesh();  
posCenter.x = topCenter.x - 3 * actualCabinWidth / 4f;  
negCenter.x = posCenter.x;  
AddRectFigXZ(6, saveCollider);
```

```
ClearMesh();  
d1 = new Vector3(actualCabinWidth - actualCabinThickness, d1.y, d1.z);  
d2 = d1;  
posCenter = new Vector3(topCenter.x, posCenter.y, topCenter.z + actualCabinWidth -  
actualCabinThickness);  
negCenter = new Vector3(posCenter.x, posCenter.y, posCenter.z + actualCabinThickness);  
AddRectFigXZ(6, saveCollider);
```

```
// Боковые стенки под окнами (перпендикулярно входу)  
ClearMesh();  
d1 = new Vector3(actualCabinThickness / 2f, d1.y, 0);  
d2 = d1;  
posCenter = new Vector3(topCenter.x - actualCabinWidth + actualCabinThickness / 2f, topCenter.y  
+ lowerOffset[1] + d1.y, topCenter.z - actualCabinWidth + actualCabinThickness);  
negCenter = new Vector3(posCenter.x, posCenter.y, topCenter.z + actualCabinWidth -  
actualCabinThickness);  
AddRectFigXZ(6, saveCollider);
```

```
ClearMesh();
```



```

posCenter.x = topCenter.x + actualCabinWidth - actualCabinThickness / 2f;
negCenter.x = posCenter.x;
AddRectFigXZ(6, saveCollider);

// Опорные балки
ClearMesh();
d1 = new Vector3(actualCabinThickness / 2f, (upperOffset[2] - lowerOffset[1]) / 2f, 0);
d2 = d1;
posCenter = new Vector3(topCenter.x - actualCabinWidth + d1.x, topCenter.y + lowerOffset[1] +
d1.y, topCenter.z - actualCabinWidth);
negCenter = new Vector3(posCenter.x, posCenter.y, posCenter.z + actualCabinThickness);
AddRectFigXZ(6, saveCollider);

ClearMesh();
posCenter.x = topCenter.x - actualCabinWidth / 2f - actualCabinThickness / 2f;
negCenter.x = posCenter.x;
AddRectFigXZ(6, saveCollider);

ClearMesh();
posCenter.x = topCenter.x + actualCabinWidth / 2f + actualCabinThickness / 2f;
negCenter.x = posCenter.x;
AddRectFigXZ(6, saveCollider);

ClearMesh();
posCenter.x = topCenter.x + actualCabinWidth - actualCabinThickness / 2f;
negCenter.x = posCenter.x;
AddRectFigXZ(6, saveCollider);

ClearMesh();
posCenter.z = topCenter.z + actualCabinWidth - actualCabinThickness;
negCenter.z = posCenter.z + actualCabinThickness;
AddRectFigXZ(6, saveCollider);

ClearMesh();

```

```

posCenter.x = topCenter.x - actualCabinWidth + d1.x;
negCenter.x = posCenter.x;
AddRectFigXZ(6, saveCollider);

//Потолок
ClearMesh();
d1 = new Vector3(actualCabinWidth, 0, actualCabinWidth);
d2 = d1;
posCenter = new Vector3(topCenter.x, topCenter.y + lowerOffset[3], topCenter.z);
negCenter = new Vector3(posCenter.x, topCenter.y + upperOffset[3], posCenter.z);
AddRectFigXY(6, saveCollider);

//Лавки
ClearMesh();
d1 = new Vector3(actualCabinWidth / 4f - actualCabinThickness, actualCabinThickness / 2f, 0);
d2 = d1;
posCenter = new Vector3(topCenter.x - 3f * actualCabinWidth / 4f, topCenter.y + lowerOffset[1]
+ 3 * (upperOffset[1] - lowerOffset[1]) / 4f - actualCabinThickness / 2f, topCenter.z - actualCabinWidth +
3f * actualCabinThickness);
negCenter = new Vector3(posCenter.x, posCenter.y, topCenter.z + actualCabinWidth -
actualCabinThickness);
AddRectFigXZ(7, saveCollider);

ClearMesh();
posCenter.x = topCenter.x + 3 * actualCabinWidth / 4f;
negCenter.x = posCenter.x;
AddRectFigXZ(7, saveCollider);

//Окна кабинки (плоскость двери)
ClearMesh();
d1 = new Vector3(actualCabinWidth / 4f - actualCabinThickness, (upperOffset[2] - lowerOffset[2])
/ 2f, 0);
d2 = d1;
posCenter = new Vector3(topCenter.x - 3f * actualCabinWidth / 4f, topCenter.y + lowerOffset[2]
+ d1.y, topCenter.z - actualCabinWidth);
negCenter = new Vector3(posCenter.x, posCenter.y, posCenter.z + actualCabinThickness);

```

```
AddRectFigXZ(8, saveCollider);
```

```
ClearMesh();
```

```
posCenter.x = topCenter.x + 3f * actualCabinWidth / 4f;
```

```
negCenter.x = posCenter.x;
```

```
AddRectFigXZ(8, saveCollider);
```

```
ClearMesh();
```

```
d1 = new Vector3(actualCabinWidth - actualCabinThickness, d1.y, d1.z);
```

```
d2 = d1;
```

```
posCenter = new Vector3(topCenter.x, posCenter.y, topCenter.z + actualCabinWidth -  
actualCabinThickness);
```

```
negCenter = new Vector3(posCenter.x, posCenter.y, posCenter.z + actualCabinThickness);
```

```
AddRectFigXZ(8, saveCollider);
```

```
//Боковые окна
```

```
ClearMesh();
```

```
d1 = new Vector3(actualCabinThickness / 2f, d1.y, 0);
```

```
d2 = d1;
```

```
posCenter = new Vector3(topCenter.x - actualCabinWidth + actualCabinThickness / 2f, topCenter.y  
+ lowerOffset[2] + d1.y, topCenter.z - actualCabinWidth + actualCabinThickness);
```

```
negCenter = new Vector3(posCenter.x, posCenter.y, topCenter.z + actualCabinWidth -  
actualCabinThickness);
```

```
AddRectFigXZ(8, saveCollider);
```

```
ClearMesh();
```

```
posCenter.x = topCenter.x + actualCabinWidth - actualCabinThickness / 2f;
```

```
negCenter.x = posCenter.x;
```

```
AddRectFigXZ(8, saveCollider);
```

```
// Создание кабинок
```

```
GameObject cabin = CreateGameObject(sIndex, eIndex, "cabin" + i, true);
```

```
cabin.GetComponent<Rigidbody>().mass = 30f;
```

```
ferrisWheel.GetComponent<Rigidbody>().mass = 200f;
```

```

cabin.transform.SetParent(cabins.transform);
for (int j = sIndex; j <= eIndex; j++)
{
    ClearSavedMesh(j);
}

Rigidbody rbC = cabin.GetComponent<Rigidbody>();
rbC.centerOfMass = new Vector3(0, 0, 0);
rbC.useGravity = true;
//JointMotor motorC = new JointMotor();
//motorC.targetVelocity = 20f;
//motorC.force = 20f;

HingeJoint hjC = cabin.AddComponent<HingeJoint>();
hjC.connectedBody = rb;
hjC.anchor = topCenter;
hjC.axis = new Vector3(0, 0, 1);

//Двери кабинки
sIndex = 6;
eIndex = 8;

GameObject cabinDoors = new GameObject();
cabinDoors.name = "cabinDoors" + i.ToString();
cabinDoors.transform.SetParent(cabin.transform);
//Левая дверь
//Панель под стеклом
ClearMesh();

d1 = new Vector3(actualCabinWidth / 4f - actualCabinThickness, (upperOffset[1] - lowerOffset[1])
/ 2f, 0);
d2 = d1;

posCenter = new Vector3(topCenter.x - actualCabinWidth / 4f, topCenter.y + lowerOffset[1] + d1.y,
topCenter.z - actualCabinWidth + actualCabinThickness);

negCenter = new Vector3(posCenter.x, posCenter.y, posCenter.z + actualCabinThickness);
AddRectFigXZ(6, saveCollider);

```

```

//Панель над стеклом
ClearMesh();

d1.y = actualCabinThickness;

d2 = d1;

posCenter.y = topCenter.y + upperOffset[2] - d1.y;

negCenter.y = posCenter.y;

AddRectFigXZ(6, saveCollider);


//Балки дверей
ClearMesh();

d1 = new Vector3(actualCabinThickness / 2f, (upperOffset[2] - lowerOffset[1]) / 2f, 0);

d2 = d1;

posCenter = new Vector3(topCenter.x - actualCabinWidth / 2f + d1.x, topCenter.y + lowerOffset[1]
+ d1.y, topCenter.z - actualCabinWidth + actualCabinThickness);

negCenter = new Vector3(posCenter.x, posCenter.y, posCenter.z + actualCabinThickness);

AddRectFigXZ(6, saveCollider);


ClearMesh();

posCenter.x = topCenter.x - d1.x;

negCenter.x = posCenter.x;

AddRectFigXZ(6, saveCollider);


//Стекло двери
ClearMesh();

d1 = new Vector3(actualCabinWidth / 4f - actualCabinThickness, (upperOffset[2] - lowerOffset[2]
- 2 * actualCabinThickness) / 2f, 0);

d2 = d1;

posCenter = new Vector3(topCenter.x - actualCabinWidth / 4f, topCenter.y + lowerOffset[2] + d1.y,
topCenter.z - actualCabinWidth + actualCabinThickness);

negCenter = new Vector3(posCenter.x, posCenter.y, posCenter.z + actualCabinThickness);

AddRectFigXZ(8, saveCollider);


GameObject cabinLeftDoor = CreateGameObject(sIndex, eIndex, "cabinLeftDoor" + i, false);
cabinLeftDoor.transform.SetParent(cabinDoors.transform);

for (int j = sIndex; j <= eIndex; j++)

```

```

{
    ClearSavedMesh(j);
}

//Правая дверь
//Панель под стеклом
ClearMesh();
d1 = new Vector3(actualCabinWidth / 4f - actualCabinThickness, (upperOffset[1] - lowerOffset[1])
/ 2f, 0);
d2 = d1;
posCenter = new Vector3(topCenter.x + actualCabinWidth / 4f, topCenter.y + lowerOffset[1] +
d1.y, topCenter.z - actualCabinWidth + actualCabinThickness);
negCenter = new Vector3(posCenter.x, posCenter.y, posCenter.z + actualCabinThickness);
AddRectFigXZ(6, saveCollider);

//Панель над стеклом
ClearMesh();
d1.y = actualCabinThickness;
d2 = d1;
posCenter.y = topCenter.y + upperOffset[2] - d1.y;
negCenter.y = posCenter.y;
AddRectFigXZ(6, saveCollider);

//Балки дверей
ClearMesh();
d1 = new Vector3(actualCabinThickness / 2f, (upperOffset[2] - lowerOffset[1]) / 2f, 0);
d2 = d1;
posCenter = new Vector3(topCenter.x + d1.x, topCenter.y + lowerOffset[1] + d1.y, topCenter.z -
actualCabinWidth + actualCabinThickness);
negCenter = new Vector3(posCenter.x, posCenter.y, posCenter.z + actualCabinThickness);
AddRectFigXZ(6, saveCollider);

ClearMesh();
posCenter.x = topCenter.x + actualCabinWidth / 2f - d1.x;
negCenter.x = posCenter.x;

```

```

AddRectFigXZ(6, saveCollider);

//Стекло двери
ClearMesh();

d1 = new Vector3(actualCabinWidth / 4f - actualCabinThickness, (upperOffset[2] - lowerOffset[2]
- 2 * actualCabinThickness) / 2f, 0);

d2 = d1;

posCenter = new Vector3(topCenter.x + actualCabinWidth / 4f, topCenter.y + lowerOffset[2] +
d1.y, topCenter.z - actualCabinWidth + actualCabinThickness);

negCenter = new Vector3(posCenter.x, posCenter.y, posCenter.z + actualCabinThickness);

AddRectFigXZ(8, saveCollider);


GameObject cabinRightDoor = CreateGameObject(sIndex, eIndex, "cabinRightDoor" + i, false);
cabinRightDoor.transform.SetParent(cabinDoors.transform);
for (int j = sIndex; j <= eIndex; j++)
{
    ClearSavedMesh(j);
}


DoorsScript sriptInstance = cabin.AddComponent<DoorsScript>();
sriptInstance.leftDoor = cabinLeftDoor;
sriptInstance.rightDoor = cabinRightDoor;
sriptInstance.doorWidth = 0.9f * actualCabinWidth / 2f;
}


saveCollider = true;
if (saveCollider)
{
    sIndex = 9;
    eIndex = 9;//8;

    // Пол кабинки
    ClearMesh();
}

```

```

        d1 = new Vector3(main.GetComponent<BoxCollider>().size.x / 2f, (-(supR + wX / 4f +
actualCabinHeight) + floorDist) / 2f, 0);

        d2 = d1;

        posCenter = new Vector3(0, d1.y, -main.GetComponent<BoxCollider>().size.z);
        negCenter = new Vector3(posCenter.x, posCenter.y, posCenter.z +
main.GetComponent<BoxCollider>().size.z / 2f);
        AddRectFigXZ(9, saveCollider);

        GameObject platform = CreateGameObject(sIndex, eIndex, "Platform", false);
        platform.transform.position = new Vector3(0, -floorDist, 0);
        platform.transform.SetParent(main.transform);
        ClearSavedMesh(sIndex);
    }
}
}

```


Приложение. Объединение нескольких Mesh в один, с сохранением исходных материалов

Дополнительно хотелось бы осветить такой момент из приведённого кода, как объединение нескольких мешей с сохранением исходных материалов объектов, основная сложность с которой приходится столкнуться в процессе реализации объединения это тот факт, что сами объекты класса Mesh не имеют описания материала как такового, он запоминается в отдельном компоненте MeshRender, который вместе с Mesh сам по себе не передаётся и возникает вопрос: каким же образом мы можем объединить сетки объектов и сохранить их материалы? В процессе работы мы смогли найти ответ на данный вопрос, он состоит в том, что для этого можно использовать сабмешы или подсети, поэтому приведём здесь объяснение:

1. Мы берём на обработку все Mesh, из которых мы хотим создать цельный объект

2. Из каждого полученного Mesh мы получаем все составляющие сетку точки и треугольники и записываем их в наш массив треугольников нового меша, причём индексы треугольников надо подогнать так, чтобы они шли по порядку (среди нескольких Mesh точно найдутся треугольники с повторяющимися индексами, поэтому нам нужно провести их перенумерацию, чтобы обеспечить порядок)

```
Mesh mesh = new Mesh();
List<Vector3> helperVerticies = new List<Vector3>();
List<int> helperTriangles = new List<int>();
List<Vector2> helperUvs = new List<Vector2>();
for (int i = startIndex; i <= endIndex; i++)
{
    int vertCount = helperVerticies.Count;
    for (int j = 0; j < subVerticies[i].Count; j++)
    {
        helperVerticies.Add(subVerticies[i][j]);
    }
    for (int j = 0; j < subTriangles[i].Count; j++)
    {
        helperTriangles.Add(subTriangles[i][j] +
vertCount);
    }
    for (int j = 0; j < subUvs[i].Count; j++)
    {
        helperUvs.Add(subUvs[i][j]);
    }
}
```

Приведённый выше код собирает все точки различных Mesh в массив helperVerticies, Полигоны (треугольники) в массив helperTriangles, и дополнительно компоненты uv в массив helperUvs, мы с uv-координатами в

данной работе никаких доп действий не воспроизводили, но в случае, если вам нужны будут адекватные uv-координаты сложного объекта их вероятнее всего нужно будет переработать.

3. Записываем полученные элементы в наш новый Mesh:

```
mesh.vertices = helperVertices.ToArray();  
mesh.triangles = helperTriangles.ToArray();  
mesh.RecalculateNormals();  
mesh.triangles = null;  
mesh.uv = helperUvs.ToArray();
```

Вы можете видеть, что после присвоения треугольников происходит их обнуление, это связано с тем, что треугольники мы будем присваивать в под сети, после раздела основной сети, но нормали мы вычислим для основной сети за счёт всех треугольников

4. Установим Флаг MeshUpdateFlags так чтобы он не обновлял имеющиеся Mesh:

```
MeshUpdateFlags flags = MeshUpdateFlags.DontValidateIndices |  
MeshUpdateFlags.DontResetBoneBounds |  
MeshUpdateFlags.DontNotifyMeshUsers |  
MeshUpdateFlags.DontRecalculateBounds;
```

5. С помощью mesh.SetIndexBufferParams объявим нашей общей Mesh количество её полигонов (треугольников):

```
mesh.SetIndexBufferParams(helperTriangles.Count,  
IndexFormat.UInt32);
```

А с помощью mesh.SetIndexBufferData установим собственно имеющиеся у сети полигоны, стартовые индексы данных, число полигонов и флаги обновления:

```
mesh.SetIndexBufferData(helperTriangles.ToArray(), 0, 0,  
helperTriangles.Count, flags);
```

6. Объявим число наших сабмешей или же подсетей:

```
mesh.subMeshCount = endIndex - startIndex + 1;
```

Здесь startIndex и endIndex – целые, неотрицательные числа, индексы материалов нашей программы (мы храним Mesh под индексами соответствующих материалов для некоторых программных оптимизаций)

7. Для каждой подсетки зададим её описание:

```

int curIndex = 0;
for(int i = startIndex; i <= endIndex; i++)
{
    SubMeshDescriptor sMD = new
SubMeshDescriptor(curIndex, subTriangles[i].Count);
    mesh.SetSubMesh(i - startIndex, sMD, flags);
    curIndex += subTriangles[i].Count;
}

```

Здесь описанием подсети выступает SubMeshDescriptor, он принимает текущий индекс, в массиве треугольников большого Mesh, и число треугольников которое стоит взять, здесь у нас фактически все треугольники одинакового материала записаны подряд, в subTriangles, поэтому мы, зная сколько треугольников у нас принадлежат тому или иному материалу можем просто сдвигать индекс начала для каждой новой подсетки.

8. Создадим GameObject к которому мы хотим присоединить этот Mesh, и с помощью него получим компоненты MeshFilter и MeshRenderer:

```

GameObject gmObj = new GameObject();
gmObj.name = name;
MeshFilter meshf = gmObj.AddComponent<MeshFilter>();
MeshRenderer mr = gmObj.AddComponent<MeshRenderer>();

```

9. Создадим список материалов для наших сабмешей:

```

Material[] helperMaterials = new Material[endIndex - startIndex + 1];
for (int i = 0; i <= endIndex - startIndex; i++)
{
    helperMaterials[i] = materials[startIndex + i];
}

```

10. Присвоим получившийся большой Mesh компоненты MeshFilter, а список материалов передадим MeshRenderer:

```

meshf.sharedMesh = mesh;
mr.materials = helperMaterials;

```

Теперь имеем объект с Mesh, разбитым на сабмешы с разными материалами. Думаю, можно модифицировать данный способ, чтобы не использовать GameObject для создания, но этот метод сработал для нас, поэтому делимся тем, что имеем.

Выводы

В ходе выполнения курсовой работы был разработан мастер для создания модели колеса обозрения с некоторыми изменяемыми параметрами, проведено исследование движения созданной модели, а также в процессе написания мастера был изучен процесс слияния нескольких мешей в один, с сохранением разных материалов